

Министерство науки и высшего образования Российской Федерации

Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Волгоградский государственный технический университет»
Институт архитектуры и строительства

Факультет архитектуры и градостроительного развития
Направление 09.03.02 Информационные системы
(специальность) и
технологии
цифровых технологий в урбанистике, архитектуре и
Кафедра строительстве
Дисциплина Анализ больших данных

Утверждаю
Зав. кафедрой Парыгин Д.С.

_____ 0 ____ .

ЗАДАНИЕ на курсовую работу (проект)

С
тудент Колядов Артём Николаевич
(фамилия, имя, отчество)

Г
руппа ИСТ-1-23
1 Разработка desktop-клиента мониторинга онлайн-трансляций
. Тема: ФДО
с интеграцией видео- и аудиоанализа

Утверждена
приказом от _____ 0 ____ . № _____
2. Срок представления работы (проекта) к
защите _____ 0 ____ .
3. Содержание расчетно-пояснительной
записки: _____

4. Перечень графического
материала: _____

5. Дата выдачи задания _____ 0 ____ г.

Руководитель работы (проекта)	_____	_____
	подпись, дата	Я.А. Трудов инициалы и фамилия
Задание принял к исполнению	_____	_____
	подпись, дата	А.Н. Колядов инициалы и фамилия

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Волгоградский государственный технический университет»
Институт архитектуры и строительства

Факультет _____ архитектуры и градостроительного развития
цифровых технологий в урбанистике, архитектуре и
Кафедра _____ строительстве

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА к курсовой работе (проекту)

по дисциплине _____ Анализ больших данных

н _____ Разработка desktop-клиента мониторинга онлайн-трансляций

а тему ФДО

_____ с интеграцией видео- и аудиоанализа

С _____

тудент _____ Колядов Артём Николаевич
(фамилия, имя, отчество)

Г _____

руппа _____ ИСТ-1-23

Руководитель работы (проекта)	_____	_____
	(подпись и дата подписания)	Я.А. Трудов (инициалы и фамилия)

Члены комиссии:

_____ (подпись и дата подписания)	_____ (инициалы и фамилия)
_____ (подпись и дата подписания)	_____ (инициалы и фамилия)
_____ (подпись и дата подписания)	_____ (инициалы и фамилия)

Нормокон
тролер

(подпись и дата подписания)

Я.А. Трудов
(инициалы и фамилия)

Волгоград 2026 г.

Содержание

Введение..... **Ошибка! Закладка не определена.**

1 Описание существующих методов анализа видеопотока **Ошибка! Закладка не определена.**

1.1 Особенности анализа онлайн-трансляций учебных занятий
..... **Ошибка! Закладка не определена.**

1.2 Методы компьютерного зрения для обработки видео..... **Ошибка! Закладка не определена.**

1.3 Нейронные сети для обнаружения объектов на изображении
..... **Ошибка! Закладка не определена.**

1.4 Сравнение существующих подходов к анализу видеотрансляций
..... **Ошибка! Закладка не определена.**

2 Проектирование системы..... **Ошибка! Закладка не определена.**

2.1 Назначение и границы разрабатываемого модуля **Ошибка! Закладка не определена.**

2.2 Архитектура и информационные потоки **Ошибка! Закладка не определена.**

2.3 Сценарии состояния видеотрансляции **Ошибка! Закладка не определена.**

2.4 Алгоритм принятия решений **Ошибка! Закладка не определена.**

2.5 Архитектура модуля видеоанализа **Ошибка! Закладка не определена.**

2.6 Структура выходного JSON . **Ошибка! Закладка не определена.**

3 Реализация модуля видеоанализа **Ошибка! Закладка не определена.**

3.1 Используемые технологии и инструменты **Ошибка! Закладка не определена.**

3.2 Структура программного модуля **Ошибка! Закладка не определена.**

3.3 Подготовка и разметка набора данных **Ошибка! Закладка не определена.**

3.4 Обучение нейросетевой модели **Ошибка! Закладка не определена.**

3.5 Реализация анализа видеозаписи **Ошибка! Закладка не определена.**

3.6 Формирование JSON-файла с результатами анализа **Ошибка! Закладка не определена.**

4 Тестирование разработанного модуля **Ошибка! Закладка не определена.**

4.1 Метрики оценки качества модели **Ошибка! Закладка не определена.**

4.2 Тестирование на типовых ситуациях трансляции **Ошибка! Закладка не определена.**

4.3 Проверка работы алгоритма по времени занятия**Ошибка!**
Закладка не определена.

4.4 Анализ результатов тестирования**Ошибка!** **Закладка не**
определена.

Заключение **Ошибка! Закладка не определена.**

Список использованных источников**Ошибка!** **Закладка не**
определена.

Введение

В настоящее время факультеты, использующие дистанционные и смешанные формы обучения, сталкиваются с необходимостью регулярно контролировать состояние онлайн-трансляций учебных занятий. При ручной организации работы оператору приходится открывать множество ссылок, проверять наличие изображения и звука, следить за ошибками подключения и своевременно реагировать на нештатные ситуации.

Такая работа становится особенно трудоемкой при одновременном проведении нескольких вечерних занятий. Если трансляции открываются в разных аудиториях, оператору необходимо быстро понимать, какие пары действительно идут, какие аудитории отменены, где отсутствует изображение, где нет звука и где требуется повторное подключение к VK Call.

Актуальность курсовой работы заключается в необходимости автоматизации клиентской части системы мониторинга трансляций ФДО. Разработанный desktop-клиент объединяет получение расписания, запуск окон трансляций, раскладку по мониторам, журналирование событий и отображение результатов видео- и аудиоанализа.

В командной разработке один участник реализует нейросетевой модуль анализа видео, второй участник разрабатывает нейросетевой модуль анализа звука. В рамках данной курсовой работы рассматривается моя часть проекта: разработка всего клиентского приложения, которое связывает расписание, интерфейс оператора, окна трансляций, захват фрагментов, интеграцию с анализаторами и вывод уведомлений.

Целью курсовой работы является разработка desktop-клиента мониторинга онлайн-трансляций ФДО, позволяющего оператору запускать выбранные VK-трансляции, контролировать их состояние и получать результаты анализа видео и звука в едином интерфейсе.

Для достижения поставленной цели необходимо решить следующие задачи:

- а) изучить процесс ручного запуска и контроля онлайн-трансляций ФДО;
- б) определить функциональные требования к клиентскому приложению оператора;
- в) спроектировать архитектуру клиента с разделением на модели, представления, контроллеры и сервисы;
- г) реализовать загрузку списка аудиторий и расписания из локального файла и VK-бота;
- д) разработать главное окно клиента, окно настроек, окно подтверждения запуска и окно уведомлений;
- е) реализовать запуск нескольких окон VK Call и автоматическую раскладку по экрану;
- ж) организовать захват фрагментов трансляций и передачу данных во внешний модуль видеоанализа;
- з) предусмотреть подключение аудиоанализатора и вывод объединенного результата проверки;
- и) выполнить тестирование ключевой логики клиента и проверить соответствие требованиям.

Объектом исследования являются процессы организации и контроля онлайн-трансляций учебных занятий ФДО.

Предметом исследования являются методы проектирования desktop-клиента, обеспечивающего запуск, мониторинг и интеграцию анализаторов состояния трансляций.

Практическая значимость работы заключается в снижении ручной нагрузки на оператора трансляций, повышении наглядности контроля и создании основы для дальнейшей интеграции нейросетевых модулей анализа видео и звука.

1 Анализ предметной области и постановка задачи

1.1 Особенности контроля онлайн-трансляций ФДО

Онлайн-трансляция учебного занятия отличается от обычной видеозаписи тем, что ее состояние изменяется в реальном времени. Преподаватель может подключиться позже, временно выключить камеру, начать демонстрацию экрана, завершить занятие раньше или столкнуться с технической ошибкой VK Call.

При ручном контроле оператор открывает ссылки на аудитории, размещает окна на мониторе и визуально проверяет состояние каждого занятия. Такой подход работает при небольшом числе аудиторий, но плохо масштабируется при одновременном запуске 10-12 трансляций.

Для ФДО важна не только сама проверка видеопотока, но и удобное рабочее место оператора. Клиент должен показывать список аудиторий, время занятий, источник расписания, статус VK-авторизации, журнал событий и результаты нейросетевых модулей. Поэтому задача разработки клиента включает как техническую интеграцию, так и проектирование удобного интерфейса.

Таблица 1 – Проблемы ручного контроля трансляций

Проблема	Проявление	Решение в клиенте
Много ссылок	Оператор открывает каждую аудиторию вручную	Единый список аудиторий и кнопка запуска выбранных трансляций
Ошибки подключения	VK Call может не подключиться с первой попытки	Повторная загрузка страницы и повторная попытка входа
Сложность наблюдения	Несколько окон занимают экран хаотично	Автоматическая раскладка окон по доступным мониторам
Неочевидные проблемы	Отсутствие видео или звука может быть замечено поздно	Передача фрагментов в модули видео- и аудиоанализа
Потеря событий	Сообщения остаются в консоли или логах	Отдельное окно уведомлений и журнал проверок

1.2 Требования к клиентскому приложению

Основным требованием к клиенту является замена ручного сценария запуска трансляций. Пользователь должен видеть список аудиторий, отмечать активные занятия, подтверждать запуск и получать набор окон VK Call без ручного копирования ссылок.

В качестве источников расписания предусмотрены локальный seed-файл со ссылками и получение актуального расписания через VK Web-сессию из диалога с ботом V505_Control. Такой подход позволяет использовать клиент как в автономном режиме, так и в режиме ежедневного получения расписания.

Клиент должен быть рассчитан на рабочий процесс оператора. Поэтому в требования включены сохранение VK-сессии, настройка интервала проверок, длительности фрагмента, FPS захвата, параметров нейросети, автозапуск по московскому времени, а также очистка временных результатов.

Таблица 2 – Основные функциональные требования

Требование	Содержание	Реализация
Загрузка аудиторий	Получение 12 VK Call ссылок из seed-файла	streams.py, schedule_service.py
Получение расписания	Запрос текущего расписания через VK Web	vk_web_schedule.py, vk_auth_window.py
Выбор аудиторий	Чекбоксы и финальное подтверждение перед запуском	main_window.py, launch_review_dialog.py
Запуск окон	Открытие отдельных окон трансляций	stream_window.py, main_controller.py
Уведомления	Отдельное окно событий, ошибок и результатов анализа	notification_window.py
Проверки	Захват фрагментов и вызов анализаторов	capture.py, analysis.py
Настройки	Изменение источника, времени запуска и параметров анализа	settings_dialog.py, config.py

1.3 Роль клиента в командном проекте

Разрабатываемая система состоит из нескольких частей. Видеоанализатор отвечает за определение визуального состояния трансляции, аудиоанализатор отвечает за проверку звуковой дорожки, а клиентская подсистема обеспечивает взаимодействие пользователя с этими модулями.

Клиент не обучает нейросеть и не подменяет логику анализа. Его задача состоит в том, чтобы получить данные трансляции, подготовить фрагмент, вызвать внешний анализатор, обработать результат и показать оператору понятное сообщение. Благодаря такому разделению каждый участник команды может развивать свою часть независимо.

Интеграционная роль клиента делает его центральной частью пользовательского сценария. Именно через клиент оператор видит расписание, запускает трансляции, проверяет состояние аудиторий и реагирует на события.

2 Проектирование desktop-клиента

2.1 Архитектура приложения

Клиент реализован на языке Python с использованием библиотеки PySide6. Для встроенного отображения VK Call используется QWebView, что позволяет открывать трансляции внутри отдельных окон приложения и не зависеть от внешнего браузера.

Архитектура построена по принципам MVC. Модели описывают доменные сущности, представления отвечают за графический интерфейс, контроллер управляет пользовательскими сценариями, а сервисы инкапсулируют работу с расписанием, VK, проверками, логированием и очисткой результатов.

Дополнительно в проекте используется blueprint-слой. Он отделяет точку регистрации desktop-клиента от основной логики приложения и делает структуру проекта более расширяемой.

Таблица 3 – Структура клиентского приложения

Каталог или файл	Назначение
translazia_client/app.py	Точка создания и запуска Qt-приложения

blueprints/desktop.py	Регистрация desktop-модуля клиента
controllers/main_controller.py	Главная orchestration-логика: расписание, окна, проверки, уведомления
views/	Окна приложения: главное окно, настройки, уведомления, VK-авторизация, трансляция
services/	Внешние операции: расписание, VK-сессия, захват, healthcheck, очистка
analysis.py	Интеграция видео- и аудиоанализаторов
layout.py	Расчет размещения окон по экранам
models.py	Модель аудитории и нормализация обозначений кабинетов

2.2 Информационные потоки

Работа клиента начинается с загрузки настроек и проверки готовности окружения. Далее клиент получает список аудиторий из выбранного источника. Если используется VK Web, приложение открывает или проверяет сохраненную VK-сессию и извлекает расписание из диалога с ботом.

После выбора аудиторий оператор подтверждает запуск. Контроллер создает окна трансляций, вычисляет их расположение, открывает отдельное окно уведомлений и запускает периодические проверки. В ходе проверки фрагменты передаются в анализаторы, а результат возвращается в основной журнал и окно уведомлений.

Таким образом, клиент выступает посредником между источником расписания, VK Call, нейросетевыми модулями и оператором.

Таблица 4 – Основные информационные потоки клиента

Поток	Источник	Получатель	Результат
Расписание	Файл или VK Web	MainController	Список StreamRoom
Команда запуска	Главное окно	MainController	Создание окон трансляций
Видеофрагмент	StreamCaptureSession	AnalysisManager	Файл для анализа
Результат проверки	Видео- и аудиоанализатор	Главное окно и уведомления	Статус, проблемы, фрагмент
Настройки	SettingsDialog	config.py	JSON-файл настроек

2.3 Пользовательский интерфейс

Главное окно клиента построено как рабочая панель оператора. В верхней части расположены логотип, название системы и основные действия: обновление расписания, запуск трансляций, открытие уведомлений, вход VK и настройки. Ниже размещены карточки состояния и таблица аудиторий.

Справа находится журнал проверок. Он предназначен для быстрого просмотра результатов анализа и ошибок. Такое расположение позволяет оператору одновременно видеть список аудиторий и состояние проверок.

На рисунке 1 показано главное окно клиента после загрузки расписания из seed-файла.

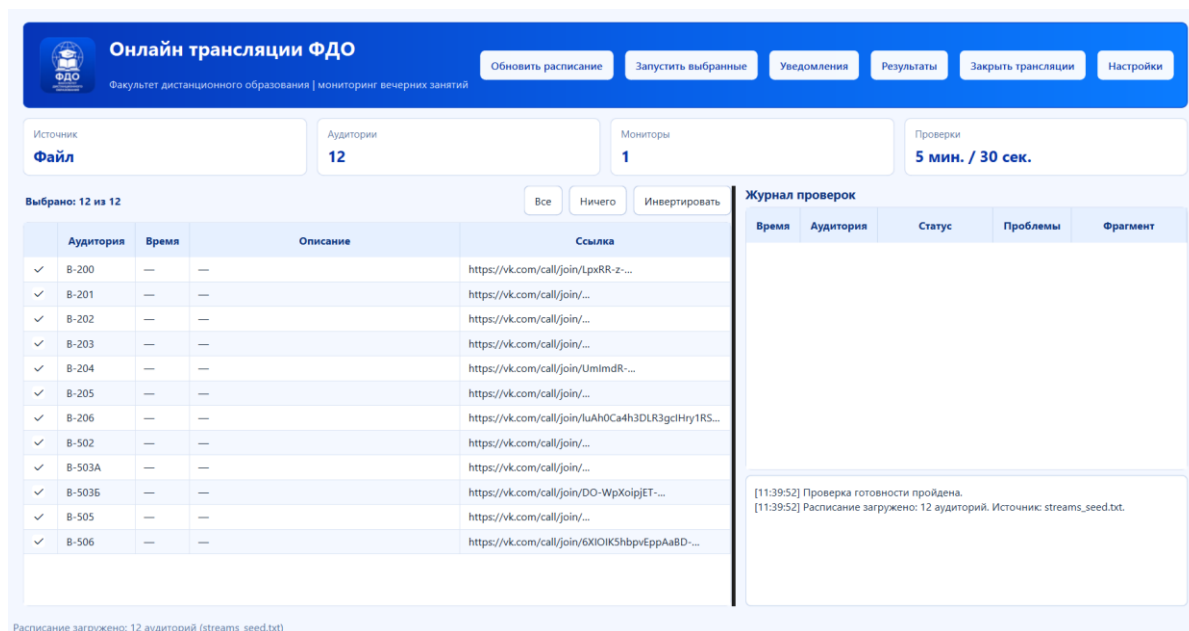


Рисунок 1 – Главное окно клиента после загрузки расписания

Окно настроек разделено на вкладки. Оператор может выбрать источник расписания, задать параметры VK, изменить время запуска, включить автозапуск и настроить параметры анализа трансляций.

На рисунке 2 показан экран настройки клиента.

Таблица 5 – Этапы загрузки расписания

Этап	Описание
Чтение настроек	Определяется режим источника: файл, VK-бот или VK Web
Получение текста	Клиент читает seed-файл либо извлекает сообщение из VK Web
Нормализация аудиторий	Обозначения кабинетов приводятся к единому виду
Сопоставление ссылок	Аудитория из расписания связывается с VK Call ссылкой
Вывод в интерфейс	Список StreamRoom отображается в таблице главного окна

3.2 Запуск и раскладка окон трансляций

После подтверждения выбранных аудиторий контроллер создает отдельное окно StreamWindow для каждой трансляции. Внутри окна используется QWebEngineView, который открывает VK Call ссылку и пытается подключиться к звонку.

Окна трансляций не размещаются случайно. Для них рассчитывается сетка с учетом количества доступных экранов и числа открываемых окон. Если подключено два монитора, окна распределяются между ними равномерно. Дополнительно учитывается место для отдельного окна уведомлений.

В окне трансляции предусмотрены элементы управления: отключение звука, обновление страницы, закрытие конкретной трансляции, фокусный режим и возврат к многооконному режиму. Это делает клиент удобным не только для массового запуска, но и для подробной проверки конкретной аудитории.

3.3 Интеграция с видео- и аудиоанализаторами

Периодическая проверка трансляции выполняется только для активных открытых окон. Если окно закрыто оператором, аудитория считается завершенной и исключается из дальнейших проверок. Такой подход снижает нагрузку и предотвращает лишние сообщения.

Модуль capture.py отвечает за получение фрагмента трансляции. Далее AnalysisManager передает видеофайл во внешний модуль vk_video_analyzer.

Результат анализа нормализуется в объект AnalysisSummary, который содержит статус, сообщение, число проблем и исходный payload.

Аудиоанализатор подключается как отдельный модуль SoundChecker. Клиент объединяет результаты видео и звука в общий статус. Если аудиопроверка недоступна, клиент формирует предупреждение, но не прерывает обработку видеорезультата.

Важно, что клиент не является нейросетью. Он выполняет инфраструктурную и пользовательскую часть: организует данные, запускает анализаторы, обрабатывает исключения и показывает результат оператору.

Таблица 6 – Интеграция клиента с анализаторами

Компонент	Ответственность	Связь с клиентом
vk_video_analyzer	Распознавание визуальных состояний и технических проблем	Вызывается из analysis.py для видеофрагмента
SoundChecker	Проверка аудиодорожки на тишину, шум и искажения	Вызывается при включенном аудиоанализе
AnalysisManager	Асинхронный запуск проверок и ограничение параллельности	Возвращает AnalysisSummary через callback
MainController	Получение результата и обновление интерфейса	Добавляет запись в журнал и окно уведомлений
NotificationWindow	Отображение событий оператору	Показывает уровень, аудиторию и сообщение

3.4 Окно уведомлений и журнал событий

Окно уведомлений является отдельной рабочей панелью. Оно открывается вместе с трансляциями и остается доступным оператору независимо от главного окна. В него попадают ошибки готовности окружения, проблемы подключения, результаты видеоанализа и аудиоанализа.

События разделяются по типам: обычные информационные сообщения, предупреждения, ошибки, видео- и аудиособытия. Это позволяет быстрее реагировать на критичные ситуации. Кроме того, окно уведомлений содержит кнопки массового отключения звука и закрытия трансляций.

На рисунке 4 показан пример журнала уведомлений.

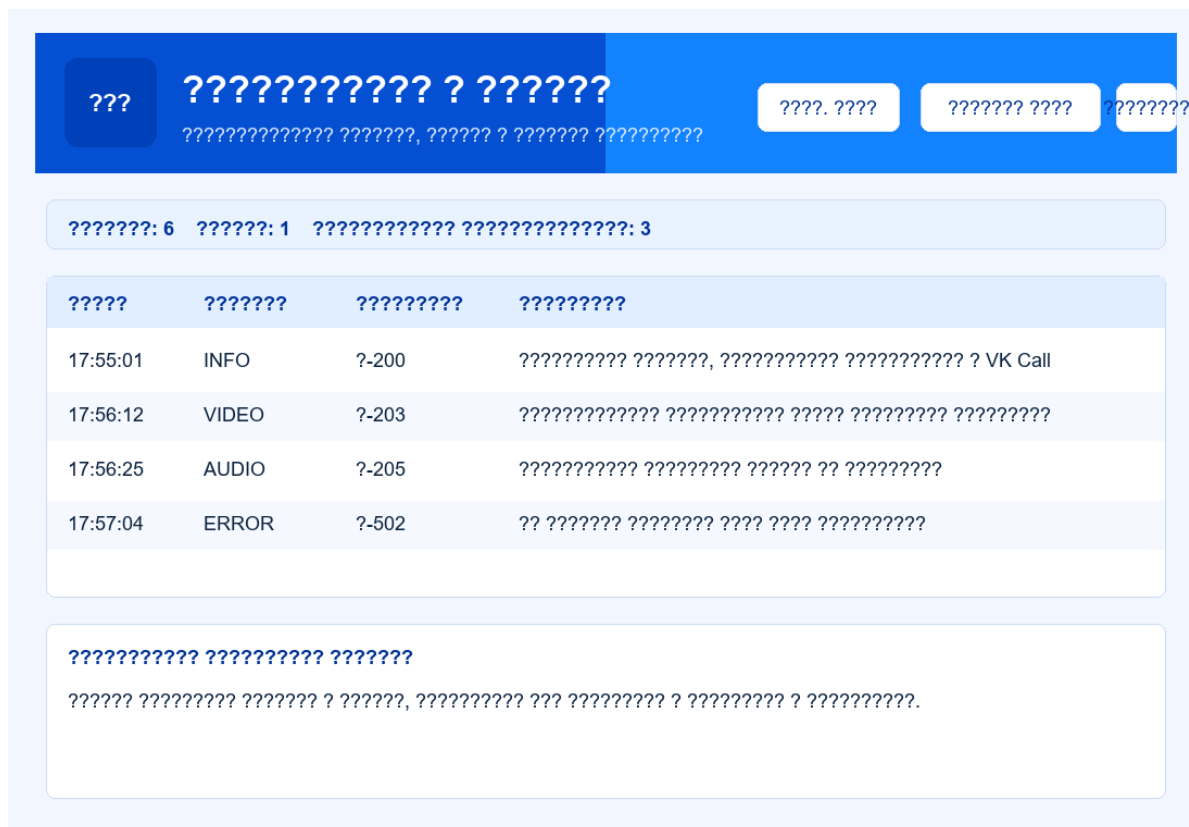


Рисунок 4 – Окно уведомлений и ошибок клиента

3.5 Настройки и хранение данных

Настройки клиента хранятся в JSON-файле. Такой формат удобен для локального desktop-приложения, так как не требует отдельной базы данных и легко читается при диагностике. В настройках выделены три группы: источник расписания, параметры запуска и параметры анализа.

Клиент также создает служебные каталоги для логов, временных фрагментов, результатов анализа и VK Web-профиля. При запуске, закрытии и по таймеру выполняется очистка временных результатов, что предотвращает накопление лишних файлов.

Для VK-авторизации используется сохраненный локальный профиль. Благодаря этому пользователю не нужно входить в VK при каждом запуске, если сессия еще действительна.

4 Тестирование и оценка результата

4.1 Модульные и поведенческие тесты

Для проверки клиентской части были подготовлены модульные и поведенческие тесты. Они не запускают реальные VK Call трансляции, но проверяют ключевую логику, которую можно надежно протестировать автоматически: парсинг аудиторий, работу seed-файла, расчет раскладки окон, наличие SPDD-артефактов и обработку времени запуска.

Автоматические тесты запускаются командой `python -m unittest discover -s tests -v`. Ожидаемый результат выполнения тестов - ОК.

Особое внимание уделено проверке расписания и раскладки окон, так как ошибки в этих частях напрямую влияют на работу оператора.

Таблица 7 – Проверяемые сценарии

Сценарий	Проверяемая логика	Тест
Парсинг аудиторий	Извлечение аудитории и VK Call ссылки из текста	test_parse_room_url_pairs
Seed-файл	Наличие 12 аудиторий в data/streams_seed.txt	test_seed_file_has_twelve_rooms
Два монитора	Равномерное распределение окон по экранам	test_layout_uses_two_screens_evenly
Окно уведомлений	Учет дополнительного окна при раскладке	test_layout_adds_notification_window_place
VK Web расписание	Извлечение аудиторий, времени и предметов из сообщения бота	test_vk_web_schedule_parser_merges_seed_links
Московское время	Корректный расчет времени запуска	test_now_moscow_timezone
SPDD	Наличие требований, canvas, prompts и verification	test_spdd_artifacts

4.2 Проверка соответствия требованиям

В результате реализации клиент соответствует основной задаче: он загружает аудитории, показывает их оператору, позволяет исключить

отмененные занятия, открывает выбранные трансляции, размещает окна на экране и отображает уведомления.

Отдельно подтверждена архитектурная структура проекта. Код не сосредоточен в одном файле, а разделен на blueprints, controllers, views, services, ui и отдельные модули анализа, моделей и раскладки.

Интеграция с нейросетевыми модулями выполнена через отдельный слой analysis.py. Это позволяет развивать видеоанализатор и аудиоанализатор независимо от интерфейса клиента.

Таблица 8 – Матрица соответствия требованиям

Требование	Реализация	Статус
Список аудиторий	Загрузка из data/streams_seed.txt и VK Web	Выполнено
Выбор перед запуском	Чекбоксы в таблице и LaunchReviewDialog	Выполнено
Открытие трансляций	StreamWindow с QWebEngineView	Выполнено
Окно уведомлений	NotificationWindow открывается вместе с трансляциями	Выполнено
Раскладка окон	compute_window_placements	Выполнено
Настройки анализа	SettingsDialog и settings.json	Выполнено
Видеоанализ	Интеграция vk_video_analyzer	Выполнено
Аудиоанализ	Подключение SoundChecker как внешнего модуля	Частично, зависит от источника аудио
Автотесты	unittest-сценарии в tests	Выполнено

4.3 Ограничения и дальнейшее развитие

Главное ограничение связано с реальными VK Call окнами. Для полноценной проверки требуется действующая VK-авторизация, доступность ссылок и корректная работа WebView. Поэтому автоматические тесты проверяют внутреннюю логику, а запуск реальных трансляций требует ручной или стендовой проверки.

Аудиоанализатор подключен на уровне клиента, однако получение звука из WebView требует отдельного аудиомаршрута или корректной

настройки ffmpeg. В дальнейшем эту часть можно усилить, чтобы аудиопроверка выполнялась так же стабильно, как видеопроверка.

Перспективами развития являются добавление end-to-end тестового режима с локальной HTML-страницей вместо VK Call, расширение сценариев автоматического восстановления подключения, экспорт отчета по итогам вечера и более подробная аналитика по аудиториям.

Заключение

В ходе выполнения курсовой работы был разработан desktop-клиент мониторинга онлайн-трансляций ФДО. Клиент предназначен для оператора, который должен быстро получать расписание, выбирать активные аудитории, запускать несколько VK-трансляций и контролировать их состояние.

В работе были рассмотрены особенности ручного контроля онлайн-занятий и сформулированы требования к клиентскому приложению. На основе этих требований спроектирована архитектура с разделением на модели, представления, контроллеры и сервисы.

Практическая реализация выполнена на Python с использованием PySide6 и QWebEngineView. В клиенте реализованы главное окно, окно настроек, окно подтверждения запуска, отдельное окно уведомлений, расчет раскладки окон, загрузка расписания из локального файла и VK Web, сохранение настроек и интеграция с внешними модулями видео- и аудиоанализа.

Проведенное тестирование подтвердило корректность ключевой логики клиента: парсинг аудиторий, наличие 12 трансляций в seed-файле, распределение окон по мониторам, учет окна уведомлений, обработку расписания VK Web и наличие SPDD-артефактов.

Таким образом, цель курсовой работы достигнута. Разработанный клиент может использоваться как центральная пользовательская часть системы автоматизированного контроля онлайн-трансляций и служит основой для дальнейшего развития видео- и аудиоаналитики.

Список использованных источников

Python Software Foundation. Python 3 Documentation [Электронный ресурс]. – Режим доступа: <https://docs.python.org/> (дата обращения: 04.06.2026).

Qt for Python. PySide6 Documentation [Электронный ресурс]. – Режим доступа: <https://doc.qt.io/qtforpython-6/> (дата обращения: 04.06.2026).

Qt Company. QWebEngineView Class Documentation [Электронный ресурс]. – Режим доступа: <https://doc.qt.io/qt-6/qwebengineview.html> (дата обращения: 04.06.2026).

OpenCV. OpenCV Documentation [Электронный ресурс]. – Режим доступа: <https://docs.opencv.org/> (дата обращения: 04.06.2026).

Ultralytics. Ultralytics YOLO Documentation [Электронный ресурс]. – Режим доступа: <https://docs.ultralytics.com/> (дата обращения: 04.06.2026).

VK. Документация для разработчиков [Электронный ресурс]. – Режим доступа: <https://dev.vk.com/ru> (дата обращения: 04.06.2026).

Python Software Foundation. json - JSON encoder and decoder [Электронный ресурс]. – Режим доступа: <https://docs.python.org/3/library/json.html> (дата обращения: 04.06.2026).

GitHub Docs. About README files [Электронный ресурс]. – Режим доступа: <https://docs.github.com/> (дата обращения: 04.06.2026).